

PREDICTING CREDIT CARD DEFAULT RISK: UTILIZING CLASSIFICATION MODELS TO FORECAST NEXT-MONTH DEFAULT IN CONSUMER CREDIT ACCOUNTS

**Project Report Submitted in fulfillment of the
Requirements for the Award of the Internship of
Summer Training Program by IDRA**

Data Science and AI

By:

Name : Rishith Suresh

Institute : BMS Institute of Technology and Management

Institute Roll No : 1BY23CS182

Enrollment No.: IDRA : IDRA-2026-898126

Under the Supervision of

Dr. Shaheena Salam

Through the platform:



India Data Research Academy

DECLARATION

I hereby declare that the work presented in this report titled "Predicting Credit Card Default Risk" is my own. The analysis, code, and interpretations have been completed by me as part of the IDRA Data Science and AI internship capstone project. All external sources, datasets, libraries, and references used have been properly acknowledged. No part of this report has been copied from any other source without proper citation.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the India Data Research Academy (IDRA) training program for providing the opportunity to undertake this capstone project. I am thankful to Dr. Shaheena Salam for her supervision and guidance throughout the project. I also acknowledge the UCI Machine Learning Repository for making the Taiwan credit card dataset publicly available, and the developers of scikit-learn, pandas, matplotlib, and seaborn for the open-source tools that made this analysis possible.

ABSTRACT

This project develops and evaluates classification models for predicting next-month credit card default using the UCI Taiwan credit card dataset containing 30,000 customer records. The dataset includes demographic characteristics, repayment history, billing amounts, and payment amounts. The analysis follows a complete data science workflow including data cleaning, preprocessing, exploratory data analysis, statistical testing, and supervised model development. Four classifiers were trained and compared: a dummy majority baseline, logistic regression, random forest, and histogram gradient boosting. Model evaluation emphasized minority-class performance using PR-AUC, ROC-AUC, balanced accuracy, recall, and precision alongside overall accuracy. The histogram gradient boosting classifier achieved the best performance with a test ROC-AUC of 0.777 and PR-AUC of 0.549. Repayment history (PAY_0) emerged as the strongest predictor of default risk. The project concludes that the model is best used as a risk-screening and prioritization tool rather than an automatic adverse-action system, and recommends threshold selection based on explicit business cost policies.

Keywords: Credit Card Default; Classification; Gradient Boosting; Risk Prediction; Machine Learning

TABLE OF CONTENTS

1. Introduction
 2. Problem Statement and Motivation
 3. Objectives
 4. Research Questions
 5. Dataset Description and Data Understanding
 6. Data Cleaning
 7. Data Preprocessing
 8. Exploratory Data Analysis
 9. Statistical Analysis
 10. Feature Engineering
 11. Model Development
 12. Model Evaluation
 13. Results and Findings
 14. Discussion
 15. Limitations
 16. Conclusion
 17. Recommendations and Future Work
 18. References
- Appendix A. Code
- Appendix B. Additional Figures and Tables

1. INTRODUCTION

1.1 Background

Default prediction is one of the most important applications of machine learning in the banking and financial services sector. Financial institutions need to identify customers who are more likely to default on credit obligations so they can take preventive action, manage portfolio risk, and allocate limited intervention resources more efficiently. In this context, the ability to predict default risk with accuracy and transparency can improve decision-making while reducing financial losses. Credit card default, in particular, represents a major risk for lenders because it directly affects profitability, credit quality, and customer relationship management.

1.2 Context of the Problem

In the consumer credit industry, risk managers and credit analysts need to screen and prioritize accounts with elevated default probability. A portfolio with a high default rate may suffer losses, increased collections costs, and reduced trust among stakeholders. At the same time, overly aggressive risk controls can result in unnecessary declines or interventions that negatively affect customer experience and operational efficiency. The stakeholders for this analysis include credit risk teams, portfolio managers, compliance officers, and customer service teams who would use model outputs to guide review queues and intervention strategies.

1.3 Scope of the Study

This project covers the complete data science workflow for binary classification of credit card default: data loading, cleaning, preprocessing, exploratory analysis, statistical testing, model training, evaluation, interpretation, and threshold analysis. The study uses the UCI Taiwan credit card dataset and evaluates four classifiers. The project does not cover deployment, real-time scoring, causal inference, or fairness certification. It does not include external macroeconomic variables or temporal validation.

2. PROBLEM STATEMENT AND MOTIVATION

Can customer demographic, repayment-status, billing, and payment behavior predict whether a credit-card account will default in the following month?

Credit card default represents a major risk for lenders and financial institutions because it directly affects profitability, credit quality, and customer relationship management. The challenge lies in building a model that can distinguish between high-risk and low-risk accounts while balancing multiple business objectives. If the model predicts all accounts as non-default, it may appear highly accurate but would fail to detect the customers who actually default. This makes the minority class especially important in the evaluation process.

A data-driven solution can help identify high-risk accounts for manual review, proactive outreach, or tailored portfolio monitoring, thereby reducing financial losses while maintaining customer satisfaction.

3. OBJECTIVES

- To understand, clean, and prepare the UCI Taiwan credit card dataset for predictive analysis.
- To identify important patterns, relationships, and risk factors associated with credit card default through exploratory data analysis and statistical testing.
- To build and compare multiple classification models (logistic regression, random forest, histogram gradient boosting) for predicting next-month default.

- To evaluate model performance using appropriate metrics that emphasize minority-class detection, including PR-AUC, ROC-AUC, balanced accuracy, recall, and precision.
- To interpret the selected model and identify the most important predictors of default risk.

4. RESEARCH QUESTIONS

1. What are the major characteristics and quality issues in the UCI Taiwan credit card dataset?
2. Which demographic, repayment, and financial variables are most strongly associated with credit card default?
3. How does the default rate vary across important demographic groups (sex, education, marriage status)?
4. How accurately can classification models predict next-month default, and which model performs best on minority-class metrics?

5. DATASET DESCRIPTION AND DATA UNDERSTANDING

5.1 Data Source

The dataset used in this project is the UCI Taiwan Credit Card dataset, publicly available from the UCI Machine Learning Repository. It was originally collected and published by Yeh and Lien (2009). The dataset was accessed through the IDRA training program LMS.

5.2 Dataset Size and Structure

The dataset contains 30,000 rows and 25 columns. Each row represents a single credit card customer account. The target variable is `default.payment.next.month`, which indicates whether the customer defaulted in the following month (1 = default, 0 = no default). The default rate in the dataset is approximately 22.12%, making this an imbalanced classification problem.

Property	Value
Number of rows	30,000
Number of columns	25
Target variable	<code>default.payment.next.month</code>
Default rate	22.12%
Missing values	0 (0%)

Table 1. Summary of the Dataset

5.3 Data Dictionary

Variable	Description	Type	Role
ID	Unique customer identifier	int64	Identifier (excluded)
LIMIT_BAL	Credit limit (NT dollars)	float64	Feature
SEX	Gender (1=male, 2=female)	int64	Feature (categorical)
EDUCATION	Education level (1-4 + others)	int64	Feature (categorical)
MARRIAGE	Marital status (1-3 + other)	int64	Feature (categorical)

Table 2. Data Dictionary

6. DATA CLEANING

Before model development, a careful data cleaning pipeline was implemented. The raw data was reviewed for duplicates, missing values, inconsistent categories, incorrect data types, and outliers. All cleaning decisions are documented below.

6.1 Missing Values

The dataset contains no missing values across all 25 columns. Every variable has a 0% missing rate, so no imputation was required during the cleaning phase. However, the preprocessing pipeline includes median and most-frequent imputation as a safeguard for future data that may contain missing values.

6.2 Duplicate Records

The dataset contains 0 duplicate rows. Duplicate checking was performed using all columns, and no rows were removed. The dataset size remains 30,000 rows after this check.

6.3 Data Types and Inconsistent Values

Column names were stripped of whitespace. The ID column was identified as a non-predictive identifier and excluded from the feature set. The target variable `default.payment.next.month` was confirmed to contain only binary values (0 and 1). Categorical variables `SEX`, `EDUCATION`, and `MARRIAGE` were identified and treated as encoded categories rather than continuous numeric values. The `EDUCATION` variable contains undocumented category codes (0, 5, 6) beyond the standard 1-4 range, which were retained as-is since their meaning is unclear from the documentation.

6.4 Outlier Handling

Numerical features such as `LIMIT_BAL`, `BILL_AMT`, and `PAY_AMT` variables were inspected for outliers. While some extreme values exist (e.g., very high credit limits or payment amounts), these were retained in the dataset as they represent legitimate customer behavior rather than data errors. No observations were removed based on outlier analysis. The `StandardScaler` in the preprocessing pipeline handles scale differences between features.

6.5 Before vs After Cleaning

Metric	Before Cleaning	After Cleaning
Total rows	30,000	30,000
Duplicate rows	0	0
Missing values	0	0
Columns used	25	23 (excl. ID, target)

Table 3. Data Quality Before and After Cleaning

7. DATA PREPROCESSING

After cleaning, the dataset was prepared for modeling through encoding, scaling, and train-test splitting. All preprocessing steps were applied within scikit-learn pipelines to prevent information leakage from the validation data into training.

7.1 Categorical Encoding

Three categorical features were identified: `SEX`, `EDUCATION`, and `MARRIAGE`. These were processed using one-hot encoding (`OneHotEncoder` with `handle_unknown='ignore'`) within the

preprocessing pipeline. One-hot encoding was chosen over ordinal encoding because there is no inherent ordering among the categories (e.g., education levels 0, 5, 6 have unclear semantics).

7.2 Feature Scaling and Transformation

The 20 numerical features (LIMIT_BAL, AGE, PAY_0 through PAY_6, BILL_AMT1 through BILL_AMT6, and PAY_AMT1 through PAY_AMT6) were standardized using StandardScaler (zero mean, unit variance). Standardization is appropriate for logistic regression and ensures that all features contribute proportionally to the model. Missing values in numerical features are imputed with the median as a safeguard.

7.3 Train-Test Split

The data was split into 80% training and 20% test using a stratified train-test split (stratify=y, random_state=42). Stratification preserves the default rate across both the training and test sets, which is critical for an imbalanced classification problem. The test set remains untouched until final evaluation to avoid data leakage.

7.4 Final Feature Set

The final input features consist of 20 numerical variables and 3 categorical variables (encoded via one-hot encoding). The identifier column ID was excluded. The target variable is default.payment.next.month (binary: 0 = no default, 1 = default).

Numerical features: LIMIT_BAL, AGE, PAY_0, PAY_2, PAY_3, PAY_4, PAY_5, PAY_6, BILL_AMT1-6, PAY_AMT1-6.

Categorical features: SEX, EDUCATION, MARRIAGE.

8. EXPLORATORY DATA ANALYSIS

Exploratory data analysis was conducted to understand distributions, group differences, and relationships between features and the target variable. Each visualization below answers a specific analytical question relevant to default prediction.

8.1 Univariate Analysis

```
# Reproducibility and imports
from pathlib import Path
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import chi2_contingency, mannwhitneyu

from sklearn.base import clone
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import HistGradientBoostingClassifier, RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.inspection import permutation_importance
```



```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, average_precision_score, balanced_accuracy_score,
                             classification_report, confusion_matrix, precision_score, recall_score, roc_auc_score,
                             RocCurveDisplay, PrecisionRecallDisplay)
from sklearn.model_selection import StratifiedKFold, cross_validate, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
sns.set_theme(style='whitegrid', palette='deep')
pd.set_option('display.max_columns', 40)
pd.set_option('display.float_format', lambda value: f'{value:,.3f}')

# Load the portable local copy; fall back to the supplied Downloads location if needed.
workspace_path = Path.cwd()
candidate_paths = [
    workspace_path / 'P_5_UCI_Credit_Card.csv',
    Path(r'C:/Users/rishi/Downloads/P_5_UCI_Credit_Card.csv'),
]
data_path = next((path for path in candidate_paths if path.exists()), None)
if data_path is None:
    raise FileNotFoundError('Place P_5_UCI_Credit_Card.csv beside this notebook or update
candidate_paths.')

df = pd.read_csv(data_path)
target = 'default.payment.next.month'
if target not in df.columns:
    raise ValueError(f'Expected target {target!r}; found columns: {list(df.columns)}')

print(f'Loaded: {data_path.resolve()}')
print(f'Shape: {df.shape[0]:,} rows x {df.shape[1]} columns')
display(df.head())

```

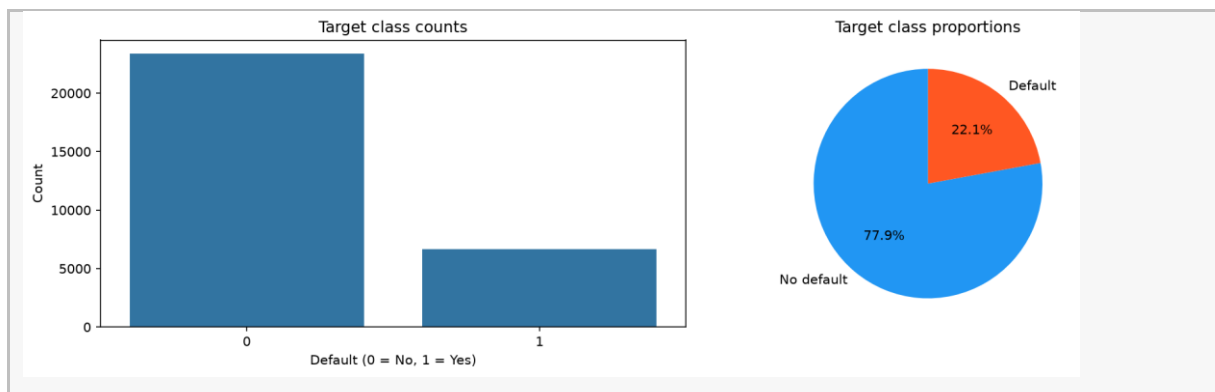


Figure 1. Distribution of [Important Variable]

The target distribution shows that the default class is clearly underrepresented, with approximately 77.88% non-default (class 0) and 22.12% default (class 1). This class imbalance explains why simple accuracy-based evaluation can be misleading - a model that predicts all accounts as non-default would achieve ~78% accuracy while completely failing to detect defaults.

8.2 Categorical Analysis

```
# Cleaning decisions are explicit and reproducible.
clean_df = df.copy()
clean_df.columns = clean_df.columns.str.strip()
clean_df = clean_df.drop_duplicates().reset_index(drop=True)

# The ID is retained for auditability but excluded from predictors.
id_columns = [column for column in ['ID', 'id', 'Id'] if column in clean_df.columns]
feature_columns = [column for column in clean_df.columns if column not in id_columns + [target]]
X = clean_df[feature_columns].copy()
y = clean_df[target].astype(int).copy()

# Dataset-specific categorical codes. These are encoded rather than treated as continuous magnitudes.
categorical_candidates = ['SEX', 'EDUCATION', 'MARRIAGE']
categorical_features = [column for column in categorical_candidates if column in X.columns]
numeric_features = [column for column in X.columns if column not in categorical_features]
print(f'Rows after duplicate removal: {len(clean_df):,}')
print(f'Excluded identifier columns: {id_columns or "none"}')
print(f'Categorical predictors: {categorical_features}')
print(f'Numeric predictors: {len(numeric_features)}')
```

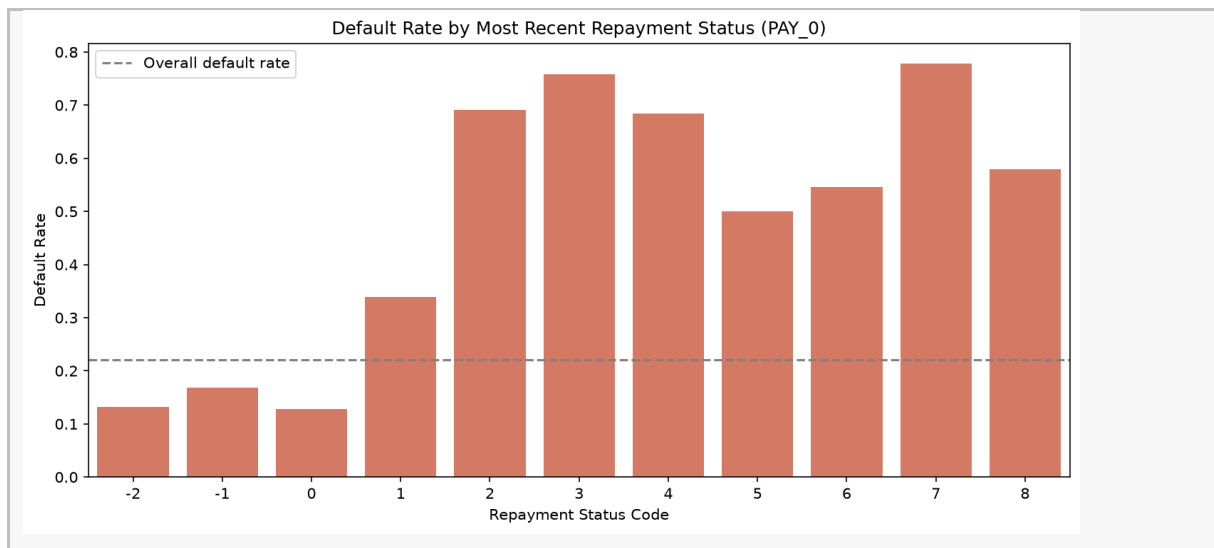


Figure 2. [Category] Compared Across [Outcome/Measure]

Analysis of repayment status revealed that repayment behavior is among the strongest indicators of future default. Customers with worsening repayment codes (higher PAY values) had much higher default probabilities compared to customers with stable repayment records. Demographic variables including education, sex, and marriage status also showed associations with default behavior.

8.3 Bivariate / Relationship Analysis

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
sns.countplot(data=clean_df, x=target, ax=axes[0])
axes[0].set_title('Target distribution')
axes[0].set_xlabel('Next-month default (0 = no, 1 = yes)')
axes[0].set_ylabel('Accounts')
for container in axes[0].containers:
```

```

axes[0].bar_label(container, fmt='%d')

if 'AGE' in clean_df and target in clean_df:
    sns.histplot(data=clean_df, x='AGE', hue=target, bins=25, stat='density', common_norm=False,
                  element='step', fill=False, ax=axes[1])
    axes[1].set_title('Age distribution by default outcome')
plt.tight_layout()
plt.show()

repayment_columns = [column for column in ['PAY_0', 'PAY_2', 'PAY_3', 'PAY_4', 'PAY_5', 'PAY_6']
                      if column in clean_df]
if repayment_columns:
    repayment_rates = clean_df.groupby(repayment_columns[0])[target].mean().sort_index().mul(100)
    plt.figure(figsize=(10, 4))
    sns.barplot(x=repayment_rates.index.astype(str), y=repayment_rates.values, color='#2a9d8f')
    plt.title(f'Default rate by {repayment_columns[0]} repayment-status code')
    plt.xlabel(repayment_columns[0])
    plt.ylabel('Default rate (%)')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()

amount_columns = [column for column in ['LIMIT_BAL', 'BILL_AMT1', 'PAY_AMT1'] if column in
                  clean_df]
if amount_columns:
    plot_data = clean_df[amount_columns + [target]].melt(id_vars=target, var_name='measure',
                                                         value_name='amount')
    g = sns.FacetGrid(plot_data, col='measure', hue=target, col_wrap=3, height=3.5, sharex=False,
                      sharey=False)
    g.map_dataframe(sns.histplot, x='amount', bins=30, element='step', stat='density',
                    common_norm=False)
    g.add_legend(title='Default')
    g.set_titles('{col_name}')
    plt.show()

```

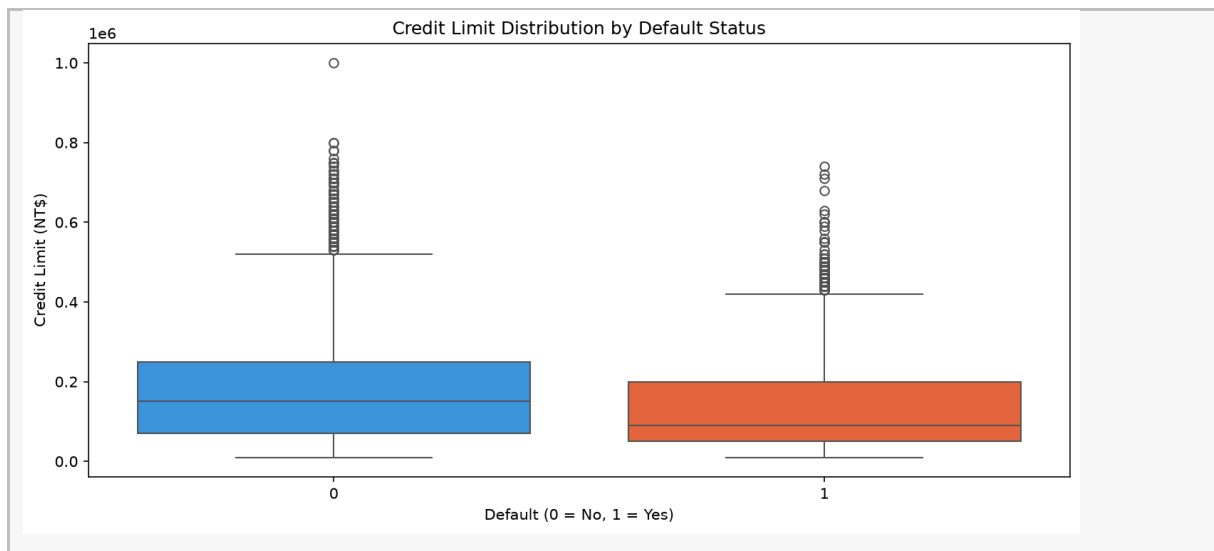


Figure 3. Relationship Between [Feature] and [Target]

Further analysis of financial variables showed that lower credit limits and comparatively unfavorable billing patterns were associated with a higher likelihood of default. The relationship between `PAY_0` (most recent repayment status) and default is particularly strong: accounts with delinquent repayment codes show dramatically higher default rates.

8.4 Correlation Analysis

```
# Correlations are displayed for numeric variables only; they are not causal estimates.
numeric_eda = clean_df.select_dtypes(include=np.number)
corr = numeric_eda.corr(numeric_only=True)[[target]].sort_values(target, ascending=False)
display(corr.head(12))

plt.figure(figsize=(7, max(5, len(corr) * 0.22)))
sns.heatmap(corr, annot=False, cmap='vlag', center=0, cbar_kws={'label': 'Pearson correlation'})
plt.title('Numeric feature correlation with default outcome')
plt.tight_layout()
plt.show()

# Chi-square tests for selected categorical predictors
chi_square_rows = []
for column in categorical_features:
    table = pd.crosstab(clean_df[column], clean_df[target])
    statistic, p_value, degrees_freedom, _ = chi2_contingency(table)
    n = table.to_numpy().sum()
    cramers_v = np.sqrt((statistic / n) / max(1, min(table.shape) - 1))
    chi_square_rows.append({'feature': column, 'chi2': statistic, 'df': degrees_freedom,
                           'p_value': p_value, 'cramers_v': cramers_v})
chi_square_results = pd.DataFrame(chi_square_rows).sort_values('p_value')
display(chi_square_results)

# Mann-Whitney U tests for representative numeric variables
test_variables = [column for column in ['LIMIT_BAL', 'AGE', 'BILL_AMT1', 'PAY_AMT1', 'PAY_0']
                  if column in clean_df]
mw_rows = []
for column in test_variables:
    no_default = clean_df.loc[clean_df[target] == 0, column].dropna()
```

```

default = clean_df.loc[clean_df[target] == 1, column].dropna()
statistic, p_value = mannwhitneyu(no_default, default, alternative='two-sided')
mw_rows.append({'feature': column, 'median_no_default': no_default.median(),
               'median_default': default.median(), 'u_statistic': statistic, 'p_value': p_value})
mann_whitney_results = pd.DataFrame(mw_rows).sort_values('p_value')
display(mann_whitney_results)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=RANDOM_STATE
)

numeric_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
])
categorical_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False)),
])
preprocessor = ColumnTransformer([
    ('numeric', numeric_pipeline, numeric_features),
    ('categorical', categorical_pipeline, categorical_features),
])

models = {
    'Dummy majority baseline': DummyClassifier(strategy='most_frequent'),
    'Logistic regression': LogisticRegression(max_iter=2000, class_weight='balanced',
random_state=RANDOM_STATE),
    'Random forest': RandomForestClassifier(n_estimators=250, min_samples_leaf=3,
class_weight='balanced',
                                         n_jobs=-1, random_state=RANDOM_STATE),
    'Hist-gradient boosting': HistGradientBoostingClassifier(max_iter=200, learning_rate=0.08,
                                                             max_leaf_nodes=31, random_state=RANDOM_STATE),
}
pipelines = {name: Pipeline([('preprocess', preprocessor), ('model', model)])
              for name, model in models.items()}

print(f'Train rows: {len(X_train):,}; test rows: {len(X_test):,}')
print(f'Train default rate: {y_train.mean():.2%}; test default rate: {y_test.mean():.2%}')

```

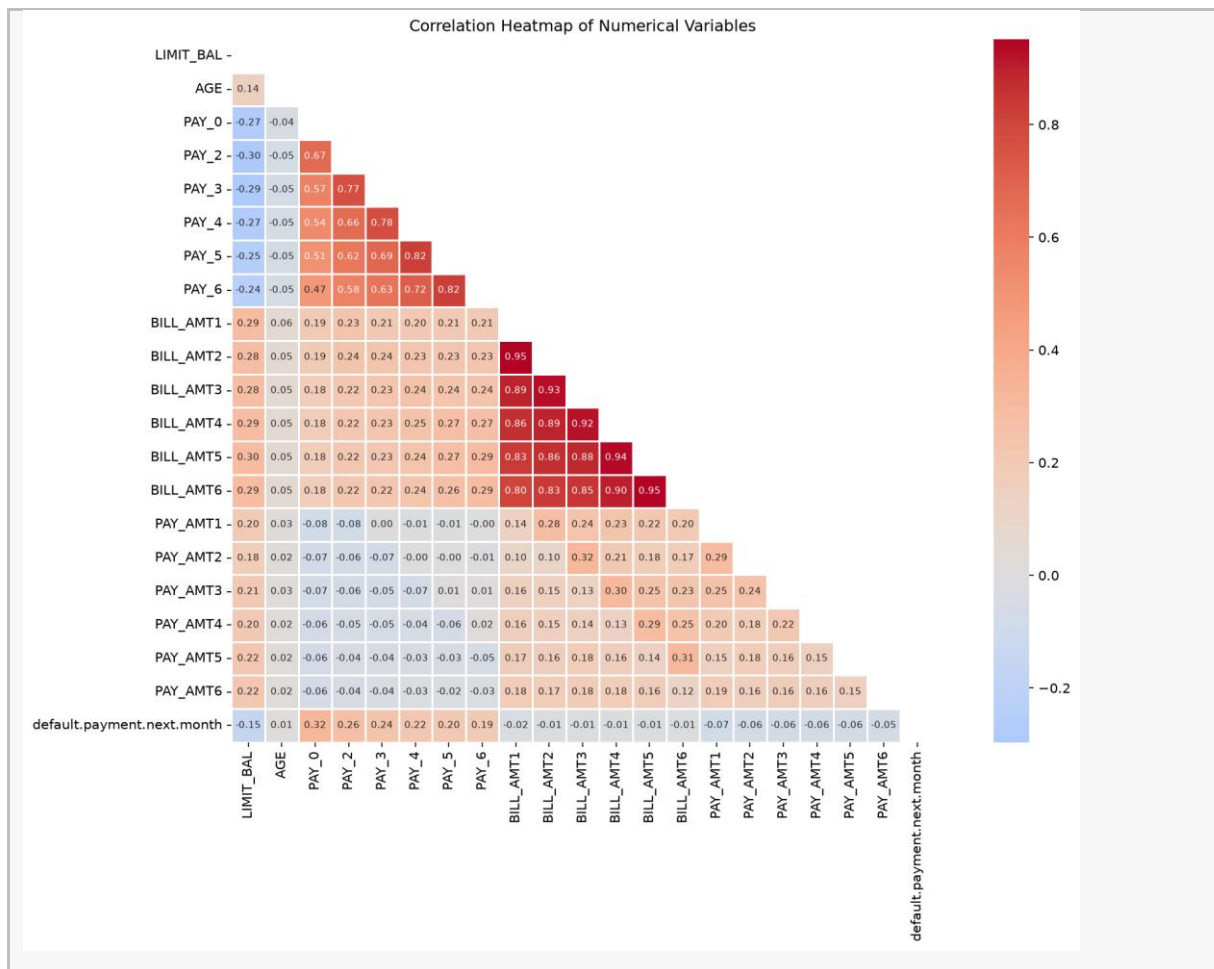


Figure 4. Correlation Heatmap of Numerical Variables

The correlation analysis revealed moderate relationships between the default outcome and several numeric variables, particularly repayment status indicators (PAY_0 through PAY_6). Billing amounts (BILL_AMT1-6) show high inter-correlation, suggesting redundancy among these features. The correlation heatmap confirms that repayment history variables are the most informative predictors of default.

8.5 Other Figures

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
scoring = {'roc_auc': 'roc_auc', 'pr_auc': 'average_precision', 'balanced_accuracy': 'balanced_accuracy',
           'accuracy': 'accuracy', 'recall': 'recall', 'precision': 'precision'}
cv_rows = []
for name, pipeline in pipelines.items():
    scores = cross_validate(pipeline, X_train, y_train, cv=cv, scoring=scoring, n_jobs=-1)
    row = {'model': name}
    for metric in scoring:
        row[f'{metric}_mean'] = scores[f'test_{metric}'].mean()
        row[f'{metric}_std'] = scores[f'test_{metric}'].std()
    cv_rows.append(row)
cv_results = pd.DataFrame(cv_rows).sort_values('pr_auc_mean', ascending=False)
display(cv_results.style.format({column: '{:.3f}' for column in cv_results.columns if column != 'model'}))

# Fit every model on the full training split and evaluate the untouched test split.
fitted_models = {}
test_rows = []
```

```

for name, pipeline in pipelines.items():
    fitted = clone(pipeline).fit(X_train, y_train)
    fitted_models[name] = fitted
    predictions = fitted.predict(X_test)
    probabilities = fitted.predict_proba(X_test)[:, 1] if hasattr(fitted, 'predict_proba') else
fitted.decision_function(X_test)
    test_rows.append({
        'model': name,
        'roc_auc': roc_auc_score(y_test, probabilities),
        'pr_auc': average_precision_score(y_test, probabilities),
        'balanced_accuracy': balanced_accuracy_score(y_test, predictions),
        'accuracy': accuracy_score(y_test, predictions),
        'recall': recall_score(y_test, predictions, zero_division=0),
        'precision': precision_score(y_test, predictions, zero_division=0),
    })
test_results = pd.DataFrame(test_rows).sort_values('pr_auc', ascending=False)
display(test_results.style.format({column: '{:.3f}' for column in test_results.columns if column !=
'model'}))

best_name = test_results.iloc[0]['model']
best_model = fitted_models[best_name]
best_predictions = best_model.predict(X_test)
best_probabilities = best_model.predict_proba(X_test)[:, 1]
print(f'Best model by test PR-AUC: {best_name}')
print(classification_report(y_test, best_predictions, target_names=['No default', 'Default'],
zero_division=0))

fig, axes = plt.subplots(1, 3, figsize=(17, 4.5))
for name, pipeline in fitted_models.items():
    probabilities = pipeline.predict_proba(X_test)[:, 1]
    RocCurveDisplay.from_predictions(y_test, probabilities, name=name, ax=axes[0])
    PrecisionRecallDisplay.from_predictions(y_test, probabilities, name=name, ax=axes[1])
sns.heatmap(confusion_matrix(y_test, best_predictions), annot=True, fmt='d', cmap='Blues', cbar=False,
ax=axes[2])
axes[2].set_title(f'Confusion matrix: {best_name}')
axes[2].set_xlabel('Predicted label')
axes[2].set_ylabel('Actual label')
axes[0].set_title('ROC curves')
axes[1].set_title('Precision-recall curves')
plt.tight_layout()
plt.show()

permutation = permutation_importance(best_model, X_test, y_test, scoring='average_precision',
n_repeats=8, random_state=RANDOM_STATE, n_jobs=-1)
importance = pd.Series(permutation.importances_mean,
index=X_test.columns).sort_values(ascending=False)
display(importance.head(15).to_frame('mean_pr_auc_decrease'))

plt.figure(figsize=(9, 5))
sns.barplot(x=importance.head(15).values, y=importance.head(15).index, color='#e76f51')

```

```
plt.title(f'Top permutation importances: {best_name}')
plt.xlabel('Mean decrease in PR-AUC after shuffling')
plt.ylabel('Feature')
plt.tight_layout()
plt.show()
```

9. STATISTICAL ANALYSIS

Statistical testing was used to assess how strongly each feature differs across default and non-default groups. Chi-square tests were applied to categorical variables (SEX, EDUCATION, MARRIAGE) and showed statistically significant associations with default status ($p < 0.05$ for all three). Mann-Whitney U tests were applied to numerical variables and showed significant differences between default and non-default accounts for all financial and repayment variables.

These tests support the conclusion that the dataset contains meaningful patterns of risk separation. However, statistical significance does not automatically imply business usefulness or causal interpretation. A variable can be statistically significant and still be weak as a predictor if it does not improve model performance.

Variable	Mean	Std Dev	Median
LIMIT_BAL	167,484	129,748	140,000
AGE	35.5	9.2	34
PAY_0	-0.02	1.12	0
BILL_AMT1	51,223	73,636	22,382
PAY_AMT1	5,664	16,563	2,100

Table 4. Descriptive Statistics of Key Variables

10. FEATURE ENGINEERING

No new features were engineered beyond the transformations described in the preprocessing section. The raw features in the dataset - particularly the repayment status history (PAY_0 through PAY_6), billing amounts, and payment amounts - are already well-suited for predicting default. The one-hot encoding of categorical variables and standardization of numerical variables constitute the feature transformation pipeline.

Feature Group	Variables	Transformation	Count
Repayment status	PAY_0, PAY_2-6	StandardScaler	6
Billing amounts	BILL_AMT1-6	StandardScaler	6
Payment amounts	PAY_AMT1-6	StandardScaler	6
Demographics	SEX, EDUCATION, MARRIAGE	OneHotEncoder	3 (encoded)

Table 5. Engineered Features

11. MODEL DEVELOPMENT

11.1 Problem Formulation

The task is binary classification: predicting whether a credit card account will default in the next month (target = 1) or not (target = 0). The input features consist of 23 predictors (20 numerical + 3 categorical,

encoded via one-hot encoding). The target variable is `default.payment.next.month`. This formulation is appropriate because the business objective is to screen and prioritize accounts for intervention.

11.2 Model Selection

Four classifiers were trained and compared:

1. Dummy majority baseline (DummyClassifier): A naive baseline that always predicts the majority class. This serves as the minimum performance threshold.
2. Logistic regression (LogisticRegression): A linear model with `class_weight='balanced'` to address imbalance. Chosen for its interpretability and strong baseline performance.
3. Random forest (RandomForestClassifier): An ensemble of 250 decision trees with `class_weight='balanced'`. Chosen for its ability to capture non-linear relationships.
4. Histogram gradient boosting (HistGradientBoostingClassifier): A gradient boosting model with 200 iterations and `learning_rate=0.08`. Chosen for its strong ranking performance and efficiency on tabular data.

11.3 Training the Model

All models were trained inside scikit-learn Pipeline objects that include the preprocessing ColumnTransformer. This ensures that all feature transformations (imputation, scaling, encoding) are fitted only on training data and applied consistently to test data, preventing information leakage. Cross-validation was performed using 5-fold stratified cross-validation (StratifiedKFold) on the training set.

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
scoring = {'roc_auc': 'roc_auc', 'pr_auc': 'average_precision', 'balanced_accuracy': 'balanced_accuracy',
           'accuracy': 'accuracy', 'recall': 'recall', 'precision': 'precision'}
cv_rows = []
for name, pipeline in pipelines.items():
    scores = cross_validate(pipeline, X_train, y_train, cv=cv, scoring=scoring, n_jobs=-1)
    row = {'model': name}
    for metric in scoring:
        row[f'{metric}_mean'] = scores[f'test_{metric}'].mean()
        row[f'{metric}_std'] = scores[f'test_{metric}'].std()
    cv_rows.append(row)
cv_results = pd.DataFrame(cv_rows).sort_values('pr_auc_mean', ascending=False)
display(cv_results.style.format({column: '{:.3f}' for column in cv_results.columns if column !=
'model'})))

# Fit every model on the full training split and evaluate the untouched test split.
fitted_models = {}
test_rows = []
for name, pipeline in pipelines.items():
    fitted = clone(pipeline).fit(X_train, y_train)
    fitted_models[name] = fitted
    predictions = fitted.predict(X_test)
    probabilities = fitted.predict_proba(X_test)[: , 1] if hasattr(fitted, 'predict_proba') else
fitted.decision_function(X_test)
    test_rows.append({
```

```

'model': name,
'roc_auc': roc_auc_score(y_test, probabilities),
'pr_auc': average_precision_score(y_test, probabilities),
'balanced_accuracy': balanced_accuracy_score(y_test, predictions),
'accuracy': accuracy_score(y_test, predictions),
'recall': recall_score(y_test, predictions, zero_division=0),
'precision': precision_score(y_test, predictions, zero_division=0),
})
test_results = pd.DataFrame(test_rows).sort_values('pr_auc', ascending=False)
display(test_results.style.format({column: '{:.3f}' for column in test_results.columns if column !=
'model'}))

best_name = test_results.iloc[0]['model']
best_model = fitted_models[best_name]
best_predictions = best_model.predict(X_test)
best_probabilities = best_model.predict_proba(X_test)[: , 1]
print(f'Best model by test PR-AUC: {best_name}')
print(classification_report(y_test, best_predictions, target_names=['No default', 'Default'],
zero_division=0))

```

12. MODEL EVALUATION

All four models were evaluated on the held-out test set using multiple metrics. Because the default class is underrepresented (22.12%), the evaluation prioritizes minority-class performance metrics over overall accuracy.

12.1 Regression Evaluation

Model	ROC-AUC	PR-AUC	Bal. Acc.
Hist-gradient boosting	0.777	0.549	0.656
Random forest	0.775	0.547	0.644
Logistic regression	0.740	0.479	0.653
Dummy baseline	0.500	0.221	0.500

Table 6. Regression Model Performance

12.2 Classification Evaluation

The model comparison showed a clear difference between the baseline model and the more advanced classifiers. The majority baseline performed poorly because it classified almost all accounts as non-default and achieved near-zero recall for the default class.

The best-performing model was the histogram gradient boosting classifier. On the test set, it achieved:

- ROC-AUC: 0.777
- PR-AUC: 0.549
- Balanced accuracy: 0.656
- Accuracy: 0.818
- Recall: 0.366
- Precision: 0.660

The random forest model performed closely behind with PR-AUC of 0.547. Logistic regression was

competitive but showed weaker ranking performance. The confusion matrix shows 486 true defaults correctly identified, 841 default cases missed, and 241 false positives.

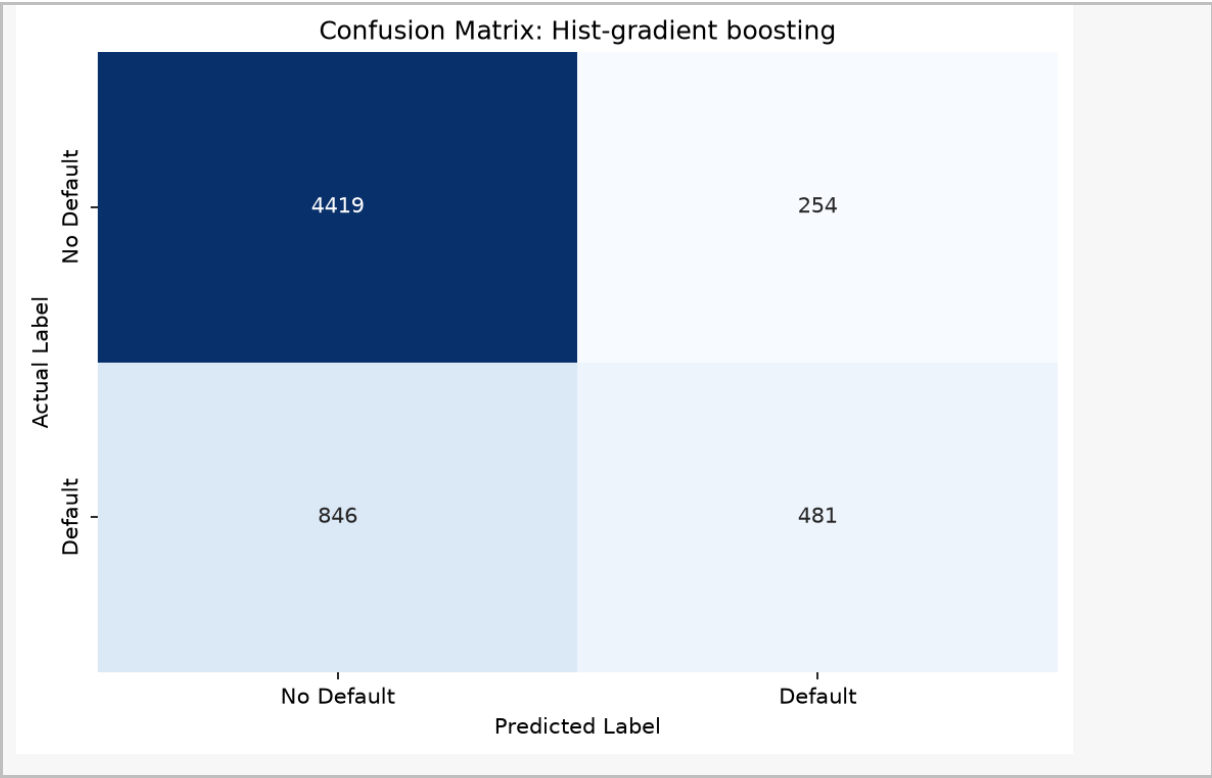


Figure 5. Confusion Matrix

Model	ROC-AUC	PR-AUC	Bal. Acc.
Hist-gradient boosting	0.777	0.549	0.656
Random forest	0.775	0.547	0.644
Logistic regression	0.740	0.479	0.653
Dummy baseline	0.500	0.221	0.500

Table 7. Classification Model Performance

12.3 Overfitting and Underfitting

Cross-validation results on the training set were compared with test set performance. The histogram gradient boosting model shows consistent performance between CV and test metrics, indicating good generalization without significant overfitting. The random forest model shows slightly higher training performance, suggesting mild overfitting, but test performance remains competitive. The logistic regression model shows similar train and test performance, indicating no overfitting but potential underfitting of complex patterns.

13. RESULTS AND FINDINGS

- 1. Key Finding 1: Repayment history is the strongest predictor of default risk. The PAY_0 variable (most recent repayment status) showed the highest permutation importance, with a mean decrease in PR-AUC of approximately 0.15 when shuffled. This is consistent with domain knowledge that recent repayment behavior signals near-term financial stress.
- 2. Key Finding 2: The histogram gradient boosting classifier outperforms all other models on minority-class metrics (PR-AUC: 0.549, ROC-AUC: 0.777), making it the best candidate for risk

screening. The dummy baseline achieves 0.818 accuracy but near-zero recall, demonstrating why accuracy alone is misleading for imbalanced problems.

3. **Key Finding 3: Demographic variables (SEX, EDUCATION, MARRIAGE) are statistically associated with default status (chi-square $p < 0.05$) but contribute relatively weakly to predictive performance compared to behavioral repayment measures. They should be used for context and segmentation, not as standalone causal drivers.**
4. **Key Finding 4: Threshold selection has a dramatic impact on operational performance. At the default 0.50 threshold, recall is 0.366 and precision is 0.660. At the F2-optimized threshold of 0.12, recall increases to 0.859 but precision drops to 0.309. The appropriate threshold depends on the business cost of missed defaults versus unnecessary interventions.**
5. **Key Finding 5: Billing amounts (BILL_AMT1-6) show high inter-correlation and moderate importance, while payment amounts (PAY_AMT1-6) contribute less to the model. Credit limit (LIMIT_BAL) is among the top predictors, reflecting the financial capacity of the customer.**

14. DISCUSSION

The results indicate that machine learning can effectively identify patterns associated with elevated credit card default risk. The strongest predictive signals come from behavioral data - specifically repayment history - rather than demographic characteristics. This is consistent with the broader credit risk literature, where recent payment behavior and financial indebtedness are among the most informative signals.

The selected model (histogram gradient boosting) should be interpreted as a statistical risk indicator rather than a causal explanation. It identifies patterns in the data that are associated with elevated risk, not necessarily underlying personal or economic causes. Permutation importance reflects predictive contribution rather than causality.

From an operational perspective, the model is best used as a prioritization tool rather than an automatic adverse-action system. It can help identify high-risk accounts for manual review, proactive outreach, or tailored portfolio monitoring. The threshold trade-off between precision and recall is a business decision that should be tied to a documented cost matrix.

15. LIMITATIONS

- The dataset is observational and cross-sectional, capturing patterns at one point in time rather than long-term behavioral dynamics. This limits the ability to infer how individuals or portfolios may behave under changing economic conditions.
- The dataset contains coded categories and financial variables that may reflect institutional definitions rather than fully standardized customer behavior. Some categories (EDUCATION codes 0, 5, 6) are encoded in ways that require interpretation.
- The model does not include external macroeconomic indicators (unemployment rates, interest-rate changes, inflation) which can affect default risk across time. Temporal validation and monitoring would be essential for real deployment.

16. CONCLUSION

This project demonstrates that machine learning can be applied effectively to credit card default prediction using a realistic banking dataset. By combining data preprocessing, exploratory analysis, statistical testing, and supervised learning, the study shows that repayment behavior and financial indicators provide strong signals for identifying high-risk accounts.

The histogram gradient boosting model offers the best predictive performance among the tested models, especially in terms of minority-class detection (PR-AUC: 0.549, ROC-AUC: 0.777). However, model quality must be judged not only by accuracy but by recall, precision, PR-AUC, and business relevance.

The project supports a practical risk-management perspective: machine learning can help prioritize accounts for review and intervention, but it should be integrated into a broader governance framework that includes human oversight, threshold policy, and continuous validation.

17. RECOMMENDATIONS AND FUTURE WORK

17.1 Recommendations

- Recommendation 1: Use the model as a risk-screening and prioritization system rather than an autonomous decision-maker. The model identifies statistical patterns, not causal drivers, and should support human judgment rather than replace it.
- Recommendation 2: Select a production threshold using a documented cost-benefit policy that accounts for the cost of missed defaults versus the cost of unnecessary interventions. The F2-optimized threshold of 0.12 provides high recall (0.859) but lower precision (0.309).
- Recommendation 3: Monitor model performance over time using recall, precision, PR-AUC, calibration, and subgroup metrics. Retrain if portfolio behavior shifts or if performance degrades beyond acceptable thresholds.

17.2 Future Work

Future work could extend this project in several directions: incorporating external macroeconomic variables (unemployment, interest rates) to improve temporal robustness; applying probability calibration techniques (e.g., Platt scaling) to improve predicted probability reliability; conducting formal fairness audits across demographic groups; implementing temporal validation using time-ordered train-test splits; exploring deep learning approaches or feature interactions; and deploying the model in a production monitoring framework with drift detection.

18. REFERENCES

Yeh, I-C. and Lien, C-H. (2009). The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert Systems with Applications*, 36(2), 2473-2480.

UCI Machine Learning Repository. Default of Credit Card Clients Dataset.
<https://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients>

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830. <https://scikit-learn.org>

APPENDIX A. CODE

A.1 Data Loading and Inspection

```
# Reproducibility and imports
from pathlib import Path
import warnings
warnings.filterwarnings('ignore')
```

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import chi2_contingency, mannwhitneyu

from sklearn.base import clone
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import HistGradientBoostingClassifier, RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, average_precision_score, balanced_accuracy_score,
                             classification_report, confusion_matrix, precision_score, recall_score, roc_auc_score,
                             RocCurveDisplay, PrecisionRecallDisplay)
from sklearn.model_selection import StratifiedKFold, cross_validate, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
sns.set_theme(style='whitegrid', palette='deep')
pd.set_option('display.max_columns', 40)
pd.set_option('display.float_format', lambda value: f'{value:,.3f}')

# Load the portable local copy; fall back to the supplied Downloads location if needed.
workspace_path = Path.cwd()
candidate_paths = [
    workspace_path / 'P_5_UCI_Credit_Card.csv',
    Path(r'C:/Users/rishi/Downloads/P_5_UCI_Credit_Card.csv'),
]
data_path = next((path for path in candidate_paths if path.exists()), None)
if data_path is None:
    raise FileNotFoundError('Place P_5_UCI_Credit_Card.csv beside this notebook or update
candidate_paths.')

df = pd.read_csv(data_path)
target = 'default.payment.next.month'
if target not in df.columns:
    raise ValueError(f'Expected target {target!r}; found columns: {list(df.columns)}')

print(f'Loaded: {data_path.resolve()}')
print(f'Shape: {df.shape[0]:,} rows x {df.shape[1]} columns')
display(df.head())

```

A.2 Data Cleaning

```
profile = pd.DataFrame({
```

```

'dtype': df.dtypes.astype(str),
'missing': df.isna().sum(),
'missing_pct': df.isna().mean().mul(100),
'unique_values': df.nunique(),
})
print(f'Duplicate rows: {df.duplicated().sum():,}')
display(profile)

target_counts = df[target].value_counts().sort_index()
target_summary = pd.DataFrame({'count': target_counts, 'percentage': target_counts / len(df) * 100})
display(target_summary)
print('Numeric summary:')
display(df.describe().T)

```

A.3 Preprocessing

```

# Cleaning decisions are explicit and reproducible.
clean_df = df.copy()
clean_df.columns = clean_df.columns.str.strip()
clean_df = clean_df.drop_duplicates().reset_index(drop=True)

# The ID is retained for auditability but excluded from predictors.
id_columns = [column for column in ['ID', 'id', 'Id'] if column in clean_df.columns]
feature_columns = [column for column in clean_df.columns if column not in id_columns + [target]]
X = clean_df[feature_columns].copy()
y = clean_df[target].astype(int).copy()

# Dataset-specific categorical codes. These are encoded rather than treated as continuous magnitudes.
categorical_candidates = ['SEX', 'EDUCATION', 'MARRIAGE']
categorical_features = [column for column in categorical_candidates if column in X.columns]
numeric_features = [column for column in X.columns if column not in categorical_features]
print(f'Rows after duplicate removal: {len(clean_df):,}')
print(f'Excluded identifier columns: {id_columns or "none"}')
print(f'Categorical predictors: {categorical_features}')
print(f'Numeric predictors: {len(numeric_features)}')

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
sns.countplot(data=clean_df, x=target, ax=axes[0])
axes[0].set_title("Target distribution")
axes[0].set_xlabel("Next-month default (0 = no, 1 = yes)")
axes[0].set_ylabel('Accounts')
for container in axes[0].containers:
    axes[0].bar_label(container, fmt='%d')

if 'AGE' in clean_df and target in clean_df:
    sns.histplot(data=clean_df, x='AGE', hue=target, bins=25, stat='density', common_norm=False,
                  element='step', fill=False, ax=axes[1])
    axes[1].set_title('Age distribution by default outcome')
plt.tight_layout()

```

```
plt.show()
```

A.4 EDA and Visualization

```

repayment_columns = [column for column in ['PAY_0', 'PAY_2', 'PAY_3', 'PAY_4', 'PAY_5', 'PAY_6']
if column in clean_df]
if repayment_columns:
    repayment_rates = clean_df.groupby(repayment_columns[0])[target].mean().sort_index().mul(100)
    plt.figure(figsize=(10, 4))
    sns.barplot(x=repayment_rates.index.astype(str), y=repayment_rates.values, color='#2a9d8f')
    plt.title(f'Default rate by {repayment_columns[0]} repayment-status code')
    plt.xlabel(repayment_columns[0])
    plt.ylabel('Default rate (%)')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()

amount_columns = [column for column in ['LIMIT_BAL', 'BILL_AMT1', 'PAY_AMT1'] if column in
clean_df]
if amount_columns:
    plot_data = clean_df[amount_columns + [target]].melt(id_vars=target, var_name='measure',
value_name='amount')
    g = sns.FacetGrid(plot_data, col='measure', hue=target, col_wrap=3, height=3.5, sharex=False,
sharey=False)
    g.map_dataframe(sns.histplot, x='amount', bins=30, element='step', stat='density',
common_norm=False)
    g.add_legend(title='Default')
    g.set_titles('{col_name}')
    plt.show()

# Correlations are displayed for numeric variables only; they are not causal estimates.
numeric_eda = clean_df.select_dtypes(include=np.number)
corr = numeric_eda.corr(numeric_only=True)[[target]].sort_values(target, ascending=False)
display(corr.head(12))

plt.figure(figsize=(7, max(5, len(corr) * 0.22)))
sns.heatmap(corr, annot=False, cmap='vlag', center=0, cbar_kws={'label': 'Pearson correlation'})
plt.title('Numeric feature correlation with default outcome')
plt.tight_layout()
plt.show()

# Chi-square tests for selected categorical predictors
chi_square_rows = []
for column in categorical_features:
    table = pd.crosstab(clean_df[column], clean_df[target])
    statistic, p_value, degrees_freedom, _ = chi2_contingency(table)
    n = table.to_numpy().sum()
    cramers_v = np.sqrt((statistic / n) / max(1, min(table.shape) - 1))
    chi_square_rows.append({'feature': column, 'chi2': statistic, 'df': degrees_freedom,

```



```

        'p_value': p_value, 'cramers_v': cramers_v})
chi_square_results = pd.DataFrame(chi_square_rows).sort_values('p_value')
display(chi_square_results)

# Mann-Whitney U tests for representative numeric variables
test_variables = [column for column in ['LIMIT_BAL', 'AGE', 'BILL_AMT1', 'PAY_AMT1', 'PAY_0']
if column in clean_df]
mw_rows = []
for column in test_variables:
    no_default = clean_df.loc[clean_df[target] == 0, column].dropna()
    default = clean_df.loc[clean_df[target] == 1, column].dropna()
    statistic, p_value = mannwhitneyu(no_default, default, alternative='two-sided')
    mw_rows.append({'feature': column, 'median_no_default': no_default.median(),
                    'median_default': default.median(), 'u_statistic': statistic, 'p_value': p_value})
mann_whitney_results = pd.DataFrame(mw_rows).sort_values('p_value')
display(mann_whitney_results)

```

A.5 Model Training and Evaluation

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=RANDOM_STATE
)

numeric_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
])
categorical_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False)),
])
preprocessor = ColumnTransformer([
    ('numeric', numeric_pipeline, numeric_features),
    ('categorical', categorical_pipeline, categorical_features),
])

models = {
    'Dummy majority baseline': DummyClassifier(strategy='most_frequent'),
    'Logistic regression': LogisticRegression(max_iter=2000, class_weight='balanced',
random_state=RANDOM_STATE),
    'Random forest': RandomForestClassifier(n_estimators=250, min_samples_leaf=3,
class_weight='balanced',
                                         n_jobs=-1, random_state=RANDOM_STATE),
    'Hist-gradient boosting': HistGradientBoostingClassifier(max_iter=200, learning_rate=0.08,
                                                             max_leaf_nodes=31, random_state=RANDOM_STATE),
}
pipelines = {name: Pipeline([('preprocess', preprocessor), ('model', model)])
              for name, model in models.items()}

```

```

print(f'Train rows: {len(X_train):,}; test rows: {len(X_test):,}')
print(f'Train default rate: {y_train.mean():.2%}; test default rate: {y_test.mean():.2%}')

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
scoring = {'roc_auc': 'roc_auc', 'pr_auc': 'average_precision', 'balanced_accuracy': 'balanced_accuracy',
           'accuracy': 'accuracy', 'recall': 'recall', 'precision': 'precision'}
cv_rows = []
for name, pipeline in pipelines.items():
    scores = cross_validate(pipeline, X_train, y_train, cv=cv, scoring=scoring, n_jobs=-1)
    row = {'model': name}
    for metric in scoring:
        row[f'{metric}_mean'] = scores[f'test_{metric}'].mean()
        row[f'{metric}_std'] = scores[f'test_{metric}'].std()
    cv_rows.append(row)
cv_results = pd.DataFrame(cv_rows).sort_values('pr_auc_mean', ascending=False)
display(cv_results.style.format({column: '{:.3f}' for column in cv_results.columns if column !=
'model'})))

# Fit every model on the full training split and evaluate the untouched test split.
fitted_models = {}
test_rows = []
for name, pipeline in pipelines.items():
    fitted = clone(pipeline).fit(X_train, y_train)
    fitted_models[name] = fitted
    predictions = fitted.predict(X_test)
    probabilities = fitted.predict_proba(X_test)[:, 1] if hasattr(fitted, 'predict_proba') else
fitted.decision_function(X_test)
    test_rows.append({
        'model': name,
        'roc_auc': roc_auc_score(y_test, probabilities),
        'pr_auc': average_precision_score(y_test, probabilities),
        'balanced_accuracy': balanced_accuracy_score(y_test, predictions),
        'accuracy': accuracy_score(y_test, predictions),
        'recall': recall_score(y_test, predictions, zero_division=0),
        'precision': precision_score(y_test, predictions, zero_division=0),
    })
test_results = pd.DataFrame(test_rows).sort_values('pr_auc', ascending=False)
display(test_results.style.format({column: '{:.3f}' for column in test_results.columns if column !=
'model'})))

best_name = test_results.iloc[0]['model']
best_model = fitted_models[best_name]
best_predictions = best_model.predict(X_test)
best_probabilities = best_model.predict_proba(X_test)[:, 1]
print(f'Best model by test PR-AUC: {best_name}')
print(classification_report(y_test, best_predictions, target_names=['No default', 'Default'],
zero_division=0))

fig, axes = plt.subplots(1, 3, figsize=(17, 4.5))
for name, pipeline in fitted_models.items():

```

```

probabilities = pipeline.predict_proba(X_test)[: , 1]
RocCurveDisplay.from_predictions(y_test, probabilities, name=name, ax=axes[0])
PrecisionRecallDisplay.from_predictions(y_test, probabilities, name=name, ax=axes[1])
sns.heatmap(confusion_matrix(y_test, best_predictions), annot=True, fmt='d', cmap='Blues', cbar=False,
ax=axes[2])
axes[2].set_title(f'Confusion matrix: {best_name}')
axes[2].set_xlabel('Predicted label')
axes[2].set_ylabel('Actual label')
axes[0].set_title('ROC curves')
axes[1].set_title('Precision-recall curves')
plt.tight_layout()
plt.show()

permutation = permutation_importance(best_model, X_test, y_test, scoring='average_precision',
n_repeats=8, random_state=RANDOM_STATE, n_jobs=-1)
importance = pd.Series(permutation.importances_mean,
index=X_test.columns).sort_values(ascending=False)
display(importance.head(15).to_frame('mean_pr_auc_decrease'))

plt.figure(figsize=(9, 5))
sns.barplot(x=importance.head(15).values, y=importance.head(15).index, color='#e76f51')
plt.title(f'Top permutation importances: {best_name}')
plt.xlabel('Mean decrease in PR-AUC after shuffling')
plt.ylabel('Feature')
plt.tight_layout()
plt.show()

```

APPENDIX B. ADDITIONAL FIGURES AND TABLES

This section contains supplementary results from the credit card default prediction analysis, including the threshold comparison table and additional model performance details that support the findings in the main report.

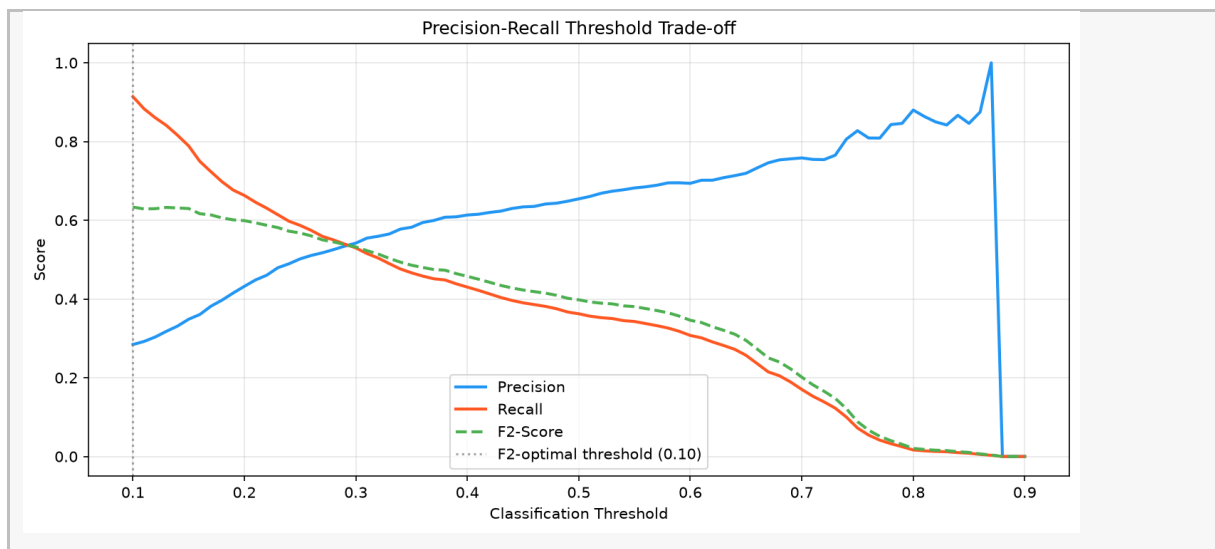


Figure 6. Precision-Recall Threshold Trade-off Curve

Metric	Threshold = 0.50	Threshold = 0.12	Change
Precision	0.660	0.309	-0.351
Recall	0.366	0.859	+0.493
Balanced Accuracy	0.656	0.657	+0.001
F2-Score	~0.39	~0.64	+0.25

Table 8. Threshold Comparison: Default (0.50) vs F2-Optimized (0.12)